

Managing the **2-second timeout window** is the single most important technical challenge in engineering your **4-Pillar Hyper-Split Payment Engine**.

Stripe Issuing real-time authorization webhooks are synchronous. When a customer taps their phone at a café POS machine, **Stripe pauses the swipe and gives your server exactly 2.00 seconds to respond** with `{"approve": true}` or `{"decline": true}`. If your server takes 2.01 seconds, Stripe will time out, use your default backup rules, and likely decline the customer.

Because you need to aggregate money from four pillars (including an external Open Banking pull), you cannot use traditional, slow database routines. You must build your tech stack specifically for ultra-low latency.

Here is exactly how to engineer your system to beat the 2-second clock every single time.

1. Ultra-Low Latency Database: Redis In-Memory Cache

Do not query a traditional database (like PostgreSQL or MySQL) to check user wallet or cashback balances during the 2-second live window. Disk-bound database queries are too slow.

- **The Architecture:** Keep all active user balances (Pillar 1: Cashback, Pillar 2: Wallet, Pillar 3: Welcome Bonus) mirrored inside a **Redis cache** (an in-memory database).
- **The Speed Advantage:** A Redis lookup takes less than **2 milliseconds**, compared to 50–100ms for a standard database query. When the webhook hits, your server can instantly read all 3 internal pillars in the blink of an eye.

2. Concurrent Execution (Parallel Async Requests)

Never run your backend logic sequentially (Step 1, then Step 2, then Step 3). If each step takes 300ms, you will easily time out.

Instead, your code must use **Asynchronous Concurrency** (e.g., `Promise.all` in Node.js, `asyncio.gather` in Python, or Goroutines in Go) to execute steps simultaneously.

```
[ WEBHOOK ARRIVES ]
    |
    ├──> (Fast Read 1): Verify Cryptographic Signature (15ms)
    ├──> (Fast Read 2): Read Pillar 1, 2, 3 balances from Redis (2ms)
    └──> (Fast Read 3): Compute the Pillar 4 "Gap" (1ms)
    |
[ EXECUTE IN PARALLEL ] ──> Fire Open Banking VRP Pull for the Gap (180ms - 350ms)
    |
[ IMMEDIATE RESPONSE ] ──> Return {"approve": true} to Stripe (Total time elapsed: ~370ms)
```



3. Optimizing the "Pillar 4 Gap" (The Open Banking Bottleneck)

The slowest part of your transaction loop will be calling your Open Banking provider (TrueLayer/Yapily) to pull the £2 gap from the customer's bank account via a Variable Recurring Payment (VRP). A network call to a high-street bank API can take anywhere from **150ms to 400ms**.

To survive this, you must implement **Optimistic Authorization with Pre-Funding**:

1. When your webhook calculates the gap, your system initiates the Open Banking VRP pull.
2. **Do not wait for the bank to finish moving the money.**
3. As long as the Open Banking API acknowledges that the transaction command was successfully *initiated without immediate error*, your backend instantly replies `{"approve": true}` to Stripe.
4. You let the bank transfer settle into your Stripe account in the background over the next few seconds. Because your system is pre-funded with your corporate operational floats inside Stripe, your account can absorb that millisecond delay easily.



4. The "Post-Swipe" Deferral (Respond Fast, Process Later)

A common mistake is trying to do everything while the customer is standing at the till. You must separate your tasks into two categories: **Synchronous** (Must happen before approving) and **Asynchronous** (Can happen after approving).

Inside the 2-Second Window (Synchronous):

- Verify Stripe's incoming signature.
- Check internal Redis balances.
- Trigger the Open Banking Gap request.
- Tell Stripe `{"approve": true}`.

Outside the 2-Second Window (Asynchronous Queue):

The millisecond you tell Stripe to approve the payment, pass the remaining business logic to a background worker queue (like BullMQ, Celery, or RabbitMQ). These tasks run calmly in the background after the user has taken their coffee:

- Executing the B2B Open Banking **20% Merchant Clawback**.
- Calculating and updating the user's new **15% Cashback wallet credit**.
- Writing permanent transaction receipts to your primary PostgreSQL database.
- Sending push notifications ("You just earned £1.50 cashback!").



5. Network Optimization & Location

Network latency (the physical distance data travels through fiber-optic cables) eats up precious milliseconds.

- Stripe's servers primarily operate out of AWS (Amazon Web Services) data centers in the **US-East (N. Virginia)** region, but they route European traffic efficiently.
- Your Open Banking providers and high-street banks operate out of the **UK-South (London)** data centers.
- **The Strategy:** Host your core backend servers on AWS in the **London (eu-west-2) region**, and utilize a Content Delivery Network (CDN) or edge computing layer to keep communication paths to Stripe and the UK banking APIs as physically short and lightning-fast as possible.